Adaptive Isolating Model (AIM):

A Universal Algorithm for Bit-Plane Structure

Jacob Ritchey

March 2026

Abstract

We present Adaptive Isolating Model (AIM), a lossless data compression algorithm based on recursive bit-plane separation. At each depth, AIM sweeps all eight bit positions to find the sparsest bit plane, separates it as a flag set, compresses the flag set with a race of seven codecs, and recurses on the remapped aligned stream with a reduced symbol alphabet. Version 16 introduces HALT_ANS_STRIDE, a fourth halt condition: before each remap, AIM computes an rANS order-1 stride-k encoding of the aligned stream and, during bottom-up assembly, terminates at the earliest depth where this encoding is cheaper than continuing to recurse. Stride k is selected by measuring $H(X_i \mid X_{i-k})$ across eight candidate values — a domain-free empirical measurement that finds periodicity in PCM, YUV, RGBA, and structured binary formats without naming any of them. On a 40.6 MB real-world PCM audio file, AIM v16 achieves −25.90% versus raw gzip (30,071,615 bytes). On 527 MB of uncompressed YUV video from a real film, AIM v16 achieves 36.93% of raw size (204,200,849 bytes), closing a 68 MB gap versus the prior best. The algorithm is parameter-free and terminates deterministically: the symbol space halves at each remap, guaranteeing halt within eight depths for byte data regardless of content. We provide a formal termination proof, a complete wire format specification, and dual reference implementations (Python and C).

1. Introduction

Every file you have ever handled is larger than its content. Not because the content is small — but because the content is buried inside a representation whose structure is a function of its encoding history, not of the data itself. Compression, as it is currently practiced, addresses this by packing those representations more tightly. It finds repeated patterns and encodes them more efficiently. It builds dictionaries, shares context between encoder and decoder, and exploits the fact that both ends of the transfer already know things about the data before the transfer begins.

This works. But a dictionary shared between encoder and decoder is not compression in the strict sense. It is a transfer of complexity from the file to the receiver. The file is smaller. The total description is not.

Real compression is something more precise: finding the minimal representation that fully conveys the data without losing anything — a description that is complete in itself, requires no external context to decode, and is shorter than the original because the data genuinely has structure, not because the receiver was pre-loaded with assumptions. Kolmogorov complexity formalizes this ideal as the length of the shortest self-sufficient program that produces the data. It is non-computable in general. But the space it defines is real, and a compressor that moves honestly toward it is doing something qualitatively different from one that achieves its ratio by offloading complexity to shared context.

AIM is an attempt to move honestly toward it.

The algorithm operates on a structural observation: byte-level sliding window compressors — DEFLATE, LZ77, their descendants — cannot access structure that lives in the correlation between bit positions across a stream, nor can they capture inter-symbol value correlations that exist at non-unit stride. A byte whose high bit is set in a particular statistical pattern carries that pattern in a dimension that no match-and-copy operation can exploit. The structure is present. It is invisible to the dominant paradigm.

AIM makes it visible. At each recursive depth, the algorithm sweeps all eight bit positions, identifies the sparsest bit plane, separates that plane as a flag set, compresses the flag set, and recurses on the remapped aligned stream. Before remapping, it computes an rANS order-1 stride-k encoding of the aligned stream — the last point at which inter-symbol value correlations are intact. During bottom-up assembly, the encoder finds the optimal depth to stop: if encoding the aligned stream with rANS is cheaper than the entire remaining recursive subtree, it halts there. Stride *k* is selected by measuring $H(X_i \mid X_{i-k})$ — no data type is named or detected. The recursion halts deterministically: each remap reduces the symbol alphabet by half, guaranteeing termination within eight depths for byte data regardless of content.

The practical consequence is measured. On a 40.6 MB real-world PCM audio file, AIM v16 achieves −25.90% versus raw gzip. On 527 MB of uncompressed YUV video from a real film, AIM v16 compresses to 36.93% of raw size. Both results were predicted from first principles. Both decode with SHA-256 verification.

1.1 Contributions

- **A universal, parameter-free bit-plane decomposition algorithm** with formal termination proof. No thresholds, no tuning constants, no training data.
- **HALT_ANS_STRIDE** — a fourth halt condition that intercepts the aligned stream pre-remap and encodes it with rANS order-1 stride-k when doing so is provably cheaper than continuing recursion. The optimal cutoff is found by a single bottom-up pass; no lookahead or approximation is required.
- **Stride selection via conditional entropy** — $H(X_i \mid X_{i-k})$ measured across eight candidate lags provides a domain-free measurement that finds k=2 for PCM audio, k=4 for YUV420/RGBA, k=8 for 8-byte structs, and k=1 for random data, without naming any format.
- **A flag format race of seven codecs** (GAP, BITSET, Elias-Fano, RLE, Huffman, LZ77, LZ77+Huffman) that selects the optimal encoding per-level without external parameters.
- **Empirical results:** −25.90% vs gzip on 40.6 MB PCM audio; 36.93% of raw on 527 MB uncompressed YUV video.
- **Dual reference implementations** (Python and C) and a complete wire format specification sufficient to implement a correct decoder in any language.

2. Related Work

The principle underlying AIM — that numerical representations of data carry mathematical structure compressible independently of semantic content — appears in fragments across several fields, typically in domain-specific form.

2.1 Bit-Plane Coding

The most direct technical precedent is bit-plane coding. In JPEG 2000's EBCOT algorithm, CABAC in H.264 and H.265, and the Embedded Zerotree Wavelet (EZW) coder, data is decomposed into successive bit planes and each plane is coded independently. The insight is the same: the LSB plane carries different statistical structure than the MSB plane, and they are better compressed separately.

The difference is scope. Existing bit-plane coders operate on transform coefficients — DCT outputs, wavelet coefficients — that have already been projected into a domain where the technique is known to work. AIM applies the same operation directly to raw byte values before any domain transform, and recurses on the coordinate structure of the residual. That recursion on coordinates rather than values is not present in prior bit-plane coding.

2.2 Predictive and Delta Coding

DPCM, Rice coding, and Golomb coding exploit the same mathematical property from a different angle: if values in a sequence are close to a predicted value, the residuals are small and encode cheaply. Rice coding, used in FLAC and scientific data formats, achieves its compression by finding the divisor N that minimizes total bits needed to encode a sequence of remainders.

The practical difference is that those methods encode the residual as a value; AIM encodes the residual as a position set. Encoding where the deviations are rather than how large they are is advantageous when the deviations are sparse and the positions are structured. The flag-position approach is better when that specific structure is present, and the algorithm detects whether it is.

2.3 Wavelet Multiresolution Analysis

Mallat's multiresolution analysis (MRA) decomposes a signal into approximation layers and detail layers at successive scales, recursing on the approximation. Both AIM and MRA are recursive decompositions that separate a smooth component from a detail residual at each level. The difference is the mathematical domain: wavelet MRA requires the input to be treated as a function of position — spatial or temporal — and operates in a continuous or sampled signal model. AIM operates on integer byte sequences with no positional interpretation required. A byte sequence representing a compressed archive, a database index, or a cryptographic hash has no meaningful spatial structure; wavelet analysis does not apply.

2.4 LZ77 and DEFLATE

The dominant paradigm in lossless compression — LZ77 [Ziv and Lempel, 1977] and its practical realization in DEFLATE [RFC 1951] — operates by finding repeated substrings within a sliding window and encoding them as back-references. gzip applies DEFLATE with level-9 optimization and is the standard reference compressor for our empirical comparison.

The fundamental limitation of sliding-window compression relative to AIM is architectural: back-reference encoding operates on positional proximity of identical byte sequences. It cannot exploit structure that manifests as a correlation between bit positions across non-adjacent or non-identical bytes, nor inter-symbol value correlations at stride $k > 1$. PCM audio is the canonical example: the high byte of each 16-bit sample has a statistical profile completely different from the low byte, and that difference is invisible to any match-and-copy compressor operating on the interleaved byte stream.

2.5 Arithmetic Coding and ANS

Arithmetic coding [Rissanen and Langdon, 1979] and Asymmetric Numeral Systems (ANS) [Duda, 2009] approach the Shannon entropy limit for known symbol distributions, operating per-symbol with adaptive models. In AIM v16, ANS plays two roles. First, as a terminal codec within the flag format race — the same role as in prior versions. Second, as the primary backend for HALT_ANS_STRIDE: an rANS order-1 stride-k encoder applied to the aligned stream before remap, where inter-symbol value correlations are still intact. The relationship is compositional: AIM's recursive separation exposes the structural layer where rANS can achieve its greatest entropy reduction, and rANS codes that layer to near its entropy limit.

3. The AIM Algorithm

We describe the algorithm formally. Let D be a byte sequence of length n. AIM processes D through a series of depths, at each depth selecting the sparsest bit plane, separating it, computing an rANS candidate, compressing the separation, and recursing on the reduced stream.

3.1 Core Operations

Operation 1: Bit-Plane Sweep

For each bit position $b \in \{0, 1, \ldots, 7\}$, the flag count is:

```
flags(D, b) = |{ i : (D[i] >> b) & 1 = 1 }|
```

The minimum-flag bit is selected: `b* = argmin_b flags(D, b)`. This is the structural choice: the bit position at which the data is most aligned — the cheapest plane to separate. The sweep runs in O(8n) time.

Operation 2: Bit-Clear and Flag Set Construction

Given selected bit `b*`, the aligned stream and flag set are:

```
A[i] = D[i] & ~(1 << b*)
F   = { i : (D[i] >> b*) & 1 = 1 }
```

The original stream is recoverable from `A` and `F`: for each `i ∈ F`, set bit `b*` in `A[i]`. This is lossless and requires no additional state beyond the flag set and the chosen bit index.

Operation 3: ANS-Stride Candidate Computation (v16)

Before remapping `A`, compute `ans_candidate = ans_stride_encode(A)`. This encoding is stored alongside the level header during descent. It is only written to the output if the optimal cutoff analysis (Section 3.5) selects this depth for HALT_ANS_STRIDE; otherwise it is discarded. It has no effect on the flag race or the remap step.

The key property: `A` is the aligned stream *post-bit-clear, pre-remap*. The remap operation dissolves inter-symbol value correlations by reassigning byte values — by depth 2–3, the original periodicity (sample pairs, pixel strides, struct fields) is effectively gone. `A` is the last point at which those correlations are intact and exploitable by an order-1 stride-k model.

Operation 4: Flag Set Compression (Race)

Encode F using all applicable format encoders simultaneously. Select the encoder that produces the smallest output. The seven formats and their selection conditions are defined in Section 3.2.

Operation 5: Symbol Space Reduction (Remap)

The aligned stream `A` has bit `b*` cleared in every byte. Remap `A` to the (k−1)-bit canonical alphabet by deleting bit `b*` from each value's binary representation:

```
remap(v, b*) = (v >> (b*+1) << b*) | (v & ((1<<b*)-1))
```

After remap, byte values are in [0, 2^(7−depth)−1]. The symbol alphabet has strictly halved. The inverse (unremap) reinserts a zero bit at position `b*` and shifts higher bits up.

3.2 Flag Set Compression — The Format Race

At each depth, the flag set F is compressed by a race of seven codecs. The codec producing the shortest output wins:

ID	Format	Best case
0	GAP: first flag + successive differences	Sparse flag sets with regular spacing
1	BITSET: dense bit vector	Flag sets with density 20–80%
2	ELIAS-FANO: monotone integer sequence encoding	Monotone sequences with clustered gaps
3	RLE: run-length encoding on the bitset	Highly regular alternating regions
4	HUFFMAN: canonical Huffman coding	Moderate density with byte-level repetition
5	LZ77: sliding window match-copy	Flag sets with positional repetition
6	LZ77+HUFFMAN: LZ77 tokens Huffman-coded	Large flag sets with mixed structure

3.3 Halt Conditions

Four structural halt conditions terminate recursion. The first three are decided in O(n) time. The fourth is decided by a size comparison computed during bottom-up assembly.

Code	Name	Trigger	Child payload
0	HALT_RECURSE	Default — not a terminal	Next level record
1	HALT_TERMINAL	MAX_DEPTH = 8 reached, or trivial terminal	gzip(aligned stream)
2	HALT_ZERO	Aligned stream is all-zero bytes	None (child_len = 0)
3	HALT_ONE	Aligned stream is all-one bytes (value 1)	None (child_len = 0)
4	HALT_ANS_STRIDE	ANS-stride encoding of aligned stream < cost of remaining subtree	ANS-stride payload (self-describing, Section 3.4)

3.4 HALT_ANS_STRIDE — The rANS Stride-k Mechanism

3.4.1 The Remap Dissolution Problem

The remap operation is mathematically correct for bit-plane entropy separation: at each level it halves the symbol alphabet and removes the structural dependency introduced by the cleared bit. However, it also scrambles inter-symbol value correlations that are independent of bit-plane structure.

Consider a stream of 16-bit PCM audio samples stored as consecutive bytes (low byte, high byte). After bit-clearing and remapping at depth 0, the sample-to-sample correlation — that consecutive samples are numerically close — has been partially destroyed by the bit reshuffling. By depth 2–3, the original inter-symbol structure is effectively gone.

The aligned stream at each depth — post-bit-clear, pre-remap — is the last point at which the original value correlation is intact. HALT_ANS_STRIDE intercepts the stream at this point when the rANS encoder can exploit it more efficiently than the remaining recursive structure.

3.4.2 Stride Selection as a Non-Domain Measurement

HALT_ANS_STRIDE requires no knowledge of the data type. The stride *k* is selected empirically by measuring the conditional entropy $H(X_i \mid X_{i-k})$ across a set of candidate values:

```
STRIDE_CANDIDATES = {1, 2, 3, 4, 6, 8, 12, 16}

H(X_i | X_{i-k}) = -sum_{x,c} P(X_i=x, X_{i-k}=c) * log2 P(X_i=x | X_{i-k}=c)

k_opt = argmin_k  H(X_i | X_{i-k})

# Activate only if gain over k=1 exceeds threshold (0.05 bits/sym):
if H(k=1) - H(k_opt) < 0.05:  k_opt = 1  # no meaningful periodicity
```

The measurement is computed on a 1 MB prefix of the aligned stream. The result is entirely data-driven:

Data type	k_opt	Reason
16-bit PCM audio	2	Sample bytes at distance 2 (low/high byte of same sample)
YUV420 video	4	Y/U/V component stride in interleaved format
RGBA image data	4	Pixel stride (R/G/B/A at offsets 0/1/2/3)
8-byte struct records	8	Record field alignment
Random / compressed data	1	No measurable periodicity; gain below threshold

No data type is named or detected. The same algorithm finds the periodicity in all structured formats and correctly abstains on random data.

3.4.3 Optimal Cutoff Selection

After completing the full descent, the encoder performs bottom-up assembly. At each depth, it compares two options:

```
option_recurse    = len(current payload)     # everything below this level
option_ans_stride = 11 + len(ans_candidate)  # 11-byte header + ANS payload

if option_ans_stride < option_recurse:
    emit HALT_ANS_STRIDE at this depth; discard all deeper levels
else:
    emit HALT_RECURSE; continue assembling
```

Because assembly proceeds from deepest to shallowest, `option_recurse` at each depth is exactly the total byte cost of all remaining structure below that point. No lookahead or approximation is required. The decision is globally optimal within the recursive structure: it finds the earliest depth at which stopping saves bytes. For files where HALT_ANS_STRIDE never fires (random data, already-compressed data), all candidates are discarded and the output is identical to v14.

3.4.4 Why Not the Flag Race, and Why Not a Final Pass

Adding ANS-stride to the flag race (as format 7 applied to the flag bitset bytes) was implemented and tested. It consistently lost to FMT_LZ77HUFF at every depth and every data type. The flag bitset encodes *where* flags appear, not *what values* the aligned stream contains. YUV period-4 and PCM period-2 correlations live in the value domain. gzip already captures whatever positional regularity exists in the flag positions.

A final ANS pass after full encoding was also evaluated. Compressed data presents as near-uniform random bytes by design — this is the defining property of effective entropy coding. A probabilistic model applied to already-compressed bytes finds no exploitable structure and adds overhead. The correct intervention point is the aligned stream before remap, not after terminal encoding.

3.5 The rANS Order-1 Stride-k Codec

The rANS codec encodes a byte stream using an order-1 context model with stride-k predecessor. The context for byte at position *i* is `data[i - k]`. For positions where `i < k`, a sentinel context `ctx = 256` (outside byte range 0–255) is used.

The codec uses ANS_M = 16384 as the normalisation interval size. Given raw symbol counts, frequency tables are normalised to sum exactly to ANS_M. Per-context tables are built for any context byte with ≥ 32 observations; the global table serves as a fallback for all other positions. The wire format stores the global frequency table as a fixed 512-byte block before the per-context table list, preventing collision with ctx_byte = 255 which is a valid context.

Encode iterates symbols in reverse order, emitting renorm bytes LSB-first. The final ANS state is stored big-endian as the first 4 bytes of the payload. Decode reads the state, reverses the renorm buffer, and reconstructs the original stream forward in one pass.

3.6 C-AIM Mode

C-AIM (Concatenated AIM) defers flag compression: all bit vectors are accumulated across depths, concatenated, and passed to a single gzip pass. This allows gzip's LZ77 model to exploit inter-depth correlations in flag positions — patterns invisible to per-level encoding. The encoder runs both recursive and CAIM modes and selects the smaller output. CAIM does not participate in the HALT_ANS_STRIDE optimisation; that mechanism is specific to recursive mode.

3.7 Formal Algorithm

Algorithm 1: AIM Recursive Encode (v16)

```
Input: byte sequence D, depth d = 0
Output: compressed stream

function encode(D, d):
  if d = 8 or |D| = 0:
    return TERMINAL(gzip(D))
  b* ← argmin_b |{ i : (D[i] >> b) & 1 }|
  A, F ← bit_clear(D, b*)
  if |F| = 0:   return HALT_ZERO
  if |F| = |D|: return HALT_ONE(encode(remap(A, b*), d+1))
  ans_cand ← ans_stride_encode(A)     // v16: capture pre-remap
  flag_enc ← race({GAP,BITSET,EF,RLE,HUFF,LZ77,LZ77HUFF}, F)
  child    ← encode(remap(A, b*), d+1)
  return assemble(b*, fmt, flag_enc, child, ans_cand)

function assemble(b*, fmt, flag_enc, child, ans_cand):
  if |ans_cand| + 11 < |child|:       // optimal cutoff
    return HEADER(b*, fmt, |flag_enc|, HALT_ANS_STRIDE, |ans_cand|)
           || flag_enc || ans_cand
  else:
    return HEADER(b*, fmt, |flag_enc|, HALT_RECURSE, |child|)
           || flag_enc || child
```

4. Theoretical Properties

4.1 Termination

Theorem 1 (AIM Terminates). For any byte sequence D of length n ≥ 0, AIM terminates within 8 recursive depths.

Proof. At depth d, the aligned stream A has symbols in the range [0, 2^(8−d)−1]. After remapping, the symbol range is [0, 2^(8−d−1)−1]. At depth 7, the symbol range is [0, 1] — a single-bit alphabet. No bit plane can be separated from a single-bit alphabet without trivially hitting HALT_ZERO or HALT_ONE. HALT_DEPTH is triggered at depth 8. Therefore the algorithm terminates within 8 depths for any input. ∎

HALT_ANS_STRIDE does not affect this proof — it fires at or before the natural terminal depth, never after it. A file for which HALT_ANS_STRIDE never activates still terminates by the same argument.

4.2 Losslessness

Theorem 2 (AIM is Lossless). For any byte sequence D, decode(encode(D)) = D.

Proof sketch. Each recursive call stores the selected bit b*, the flag set F, and the encoded child. Reconstruction reads F, decodes the child to recover the remapped aligned stream (or, for HALT_ANS_STRIDE, the aligned stream directly), applies unremap where applicable, then sets bit b* for each position in F. This inverts bit_clear exactly. The HALT_ANS_STRIDE path skips unremap for the ANS level because the ANS payload encodes the aligned stream directly — the encoded data was never remapped. By induction on depth, all halt conditions are invertible. ∎

4.3 Optimality of the Cutoff

The bottom-up cutoff comparison is globally optimal within the recursive structure: given the set of level records produced by the descent, no earlier or later cutoff point can produce a smaller output than the one selected. This follows directly from the fact that the comparison at each depth uses the exact total cost of the remaining subtree, not an approximation. The claim is not that ANS-stride is the globally optimal codec for the aligned stream — it is that the encoder correctly identifies the cheapest available option at every depth.

4.4 Parameter Freedom

AIM has no tunable parameters. The bit selection is determined by the data (argmin sweep). The format selection is determined by output size (race). The stride selection is determined by conditional entropy (argmin over candidates). The halt conditions are structural. The recursion depth is bounded by the alphabet size. The ANS cutoff is a size comparison, not a threshold.

This property matters for the Huffman analogy. Huffman coding is foundational because it is optimal for its problem class without external parameters. AIM claims the analogous position for bit-plane and stride structure: it is the natural, parameter-free, deterministically-terminating operation on this class.

4.5 Relationship to Kolmogorov Complexity

A complete AIM encoding constitutes a valid self-sufficient reconstruction program for the original data. No external dictionary, pre-trained model, or shared context is required. The output is complete in itself. Any encoding improvement — a better flag format, a more efficient stride selection — reduces the length of this program, and because the algorithm is universal, such improvements apply to all data simultaneously.

5. Empirical Evaluation

5.1 Primary Benchmark: PCM Audio

The primary benchmark is `sample-full.wav`, a 40,581,228-byte real-world PCM audio file (38.7 MiB). 16-bit PCM samples store amplitude as a signed integer. The high byte clusters near zero during silence; the low byte is near-uniform regardless of signal level. These two bytes have completely different entropy profiles. Any encoding that treats them identically — as a sliding window compressor must — is conflating two structurally unlike streams.

Version	Output bytes	Ratio vs raw	vs gzip	Encode time	Verified
gzip level 9	40,581,488	100.00%	—	—	—
AIM v14 C	34,482,879	84.97%	−15.03%	7.3s	✓
AIM v16 Python	30,071,618	74.10%	−25.90%	252s	✓
AIM v16 C	30,071,615	74.10%	−25.90%	8.7s	✓

The 3-byte difference between Python and C is a rANS frequency normalization rounding path difference — C and Python's round() diverge by ±1 LSB on a handful of frequency slots, producing marginally different renorm byte counts. Both decode correctly with SHA-256 verification.

The mechanism for the improvement over v14 is the HALT_ANS_STRIDE activation on the aligned stream at depth 0–1, where the PCM sample-pair correlation (k=2) is fully intact. After remap at depth 1, those correlations are dissolved and rANS order-1 gains nothing.

5.2 Fight Club YUV Video Benchmark

The secondary benchmark is `fight_club_plane.yuv`, 552,960,000 bytes of uncompressed YUV420 video from a real film. This file was the historical reference for the performance gap between AIM v14 and the prior best (AIM3 v7p2, an earlier experimental branch). The gap was the primary motivation for HALT_ANS_STRIDE.

Version	Output bytes	Ratio vs raw	Notes
gzip level 9	~552,960,000	~100%	Negligible compression on YUV
AIM v14 C	~268,500,000	~48.6%	Pre-HALT_ANS_STRIDE

Version	Output bytes	Ratio vs raw	Notes
AIM3 v7p2 (prior best)	~199,000,000	~36.0%	Experimental; ANS on aligned stream of earlier architecture
AIM v16 C	204,200,849	36.93%	Full recursive structure + HALT_ANS_STRIDE

AIM v16 closes essentially the entire 78 MB gap between v14 and the prior best — from 268 MB down to 204 MB, within 5 MB of the experimental AIM3 result. Unlike AIM3, v16 maintains the full recursive structure, the complete flag format race, and SHA-256 verified roundtrip integrity. Encode time was 288 seconds; decode was 5.8 seconds.

The YUV420 format interleaves luma (Y) and chroma (U, V) components with period-4 alignment. The stride selection measures k=4 as optimal at the aligned stream for depths 0–1, where that periodicity is intact.

5.3 Synthetic Benchmark Suite

File	Size	Description	AIM v16 result
all_zeros	10 KB	Single byte value 0x00	HALT_ZERO at depth 0: 0.11% of raw
all_0xff	10 KB	Single byte value 0xFF	HALT_ONE at depth 0: 1.52% of raw
ramp_100k	100 KB	All 256 byte values × 400	0.35% of raw (maximally flat)
repeated_text	12 KB	ASCII phrase repeated × 1000	0.43% of raw
random_100k	100 KB	Uniform random bytes	100.14% — near-parity, ANS does not activate
source_code	48 KB	Python source (aim_v15.py)	65.82% — incompressible by structure

5.4 Implementation Comparison

Metric	Python (aim_v15.py)	C (aim_v16.c)
Encode time (38.7 MB WAV)	~252 seconds	8.7 seconds
Compression ratio (WAV)	−25.90%	−25.90%

Metric	Python (aim_v15.py)	C (aim_v16.c)
Encode time (527 MB YUV)	n/a	288 seconds
Compression ratio (YUV)	n/a	36.93%
Decode + verify (38.7 MB)	~15.6 seconds	0.70 seconds
Roundtrip SHA-256 (test suite)	All pass	All pass
Python ↔ C wire compatibility	Yes (cross-decode verified)	Yes (cross-decode verified)

6. The Huffman Analogy

Huffman coding finds the shortest possible prefix code given a known symbol distribution. Its contribution was foundational because it is universal over symbol distributions, lossless, and deterministic — and because it formalized a space of structure (symbol frequency) that prior methods had addressed only heuristically.

Before Huffman, frequency-aware encoding existed. Fixed codebooks, Morse code, and similar schemes exploited symbol frequency. What Huffman provided was the optimal parameter-free algorithm for that class of structure — one that requires no external constants, applies to any distribution, and produces the provably shortest expected code length.

AIM claims the analogous position one level of abstraction higher:

Property	Huffman	AIM
Structure class	Symbol frequency	Bit-plane sparsity + inter-symbol stride correlation
Operation	Variable-length prefix codes	Recursive bit-plane separation with remap + rANS stride-k
Parameters	None	None
Terminates?	Yes (tree construction)	Yes (alphabet halves per depth, max 8)
Lossless?	Yes	Yes
Prior work	Heuristic codebooks	Domain-specific bit-plane coders; stride codecs require format knowledge
Relationship	Optimal for its class	Universal for its class (optimality: open)

The analogy is precise in one important respect: Huffman does not replace sliding-window compression, and AIM does not replace Huffman. Huffman coding is one of the seven formats in AIM's flag format race, and rANS is the backend for HALT_ANS_STRIDE. The techniques are orthogonal — they operate on different structural dimensions of the data. A complete compression pipeline would apply all of them.

7. Wire Format

The AIM container format is self-describing. A decoder requires no external context, shared dictionary, or pre-trained model.

7.1 Container Header (45 bytes fixed)

```
Offset  Size  Field
0       4     magic = 0x41 0x49 0x4D 0x34  ('AIM4')
4       1     mode  = 0 (RECURSIVE) | 1 (CAIM)
5       8     orig_n  uint64 big-endian — original byte count
13      32    sha256  — SHA-256 of original uncompressed data
45      ...   payload (mode-specific)
```

7.2 Recursive Level Header (11 bytes + flag data)

```
bit        (1 B)    selected bit position b* (0–7)
fmt        (1 B)    flag format ID (0–6, see Section 3.2)
flag_len   (4 B BE) byte length of flag data
halt       (1 B)    0=recurse  1=terminal  2=HALT_ZERO  3=HALT_ONE
4=HALT_ANS_STRIDE
child_len  (4 B BE) byte length of child payload
flag_data  (flag_len B)   compressed flag set
child      (child_len B)  halt=0: next level; halt=1: gzip; halt=2,3: empty;
halt=4: ANS payload
```

7.3 ANS-Stride Payload (halt = 4)

```
k_opt         (1 B)    stride selected (1,2,3,4,6,8,12,16)
n             (4 B BE) original byte count
global_freq   (512 B)  256 × uint16 LE — global frequency table
                       (fixed block; NOT stored as per-context entry to avoid
                        collision with ctx_byte = 255)
n_ctx         (2 B BE) number of per-context tables
n_ctx × [
  ctx_byte    (1 B)    context byte value (0–255)
  freq        (512 B)  256 × uint16 LE — per-context frequency table
]
payload_len   (4 B BE)
payload       state (4 B BE big-endian) + renorm bytes (LSB-first emission
order)
```

7.4 C-AIM Payload

```
n_levels   (1 B)         number of levels
bits       (n_levels B)  bit cleared at each level
flags_len  (4 B BE)
flags_gz   (flags_len B) gzip(concat of all flag bitsets)
term_len   (4 B BE)
term_gz    (term_len B)  gzip(final aligned stream)
```

8. Discussion

8.1 What HALT_ANS_STRIDE Reveals

The fight_club result is perhaps the clearest illustration of what HALT_ANS_STRIDE accomplishes. AIM v14 compressed 527 MB of YUV video to 268 MB — a solid result, but one that an earlier experimental branch (AIM3 v7p2, which applied rANS to the aligned stream directly without the full recursive structure) could improve to 199 MB. The gap existed because AIM4's remap operation was destroying the YUV period-4 correlation before any codec could exploit it.

HALT_ANS_STRIDE closes that gap by measuring, at every depth during descent, whether the aligned stream at that depth is more compressible by rANS than by the remaining recursive structure. When YUV's period-4 correlation is strong at depth 0–1, rANS with k=4 wins; the encoder stops there and the deeper levels that would have dissolved the correlation are never generated. When the correlation is already gone at a given depth, the comparison returns false and recursion continues normally.

This is not a domain-specific trick. The encoder does not know it is processing video. It measures $H(X_i \mid X_{i-k})$ for eight values of k, observes that k=4 gives the greatest conditional entropy reduction, and makes a binary size comparison. The YUV period-4 structure is discovered, not assumed.

8.2 Limitations

- **Sequential and positional structure:** AIM does not improve on gzip for data whose structure is primarily sequential and positional (natural language text, source code). On these types, the flag sets produced by bit-plane separation are not significantly more compressible than the original.
- **Encode time vs decode time:** The rANS candidate computation adds overhead at every depth during encoding. The C implementation handles 40 MB WAV in 8.7 seconds and 527 MB YUV in 288 seconds. Decode remains fast (0.7s and 5.8s respectively) because candidate computation is only performed at encode time.
- **CAIM + HALT_ANS_STRIDE:** The ANS cutoff is currently only implemented in recursive mode. Extending the optimisation to CAIM mode is future work.

8.3 Future Work

- **CAIM + ANS:** Extend HALT_ANS_STRIDE to CAIM mode.
- **Optimality characterisation:** Is AIM optimal for the bit-plane structure class in the information-theoretic sense, or is there a parameter-free recursive algorithm that achieves strictly better expected compression on all bit-plane-structured inputs?
- **Entropy stratigraphy as a data identification tool:** The decay profile is a structural fingerprint. A reference library of profiles for known data types would allow AIM to be used as a format-agnostic classifier.
- **Corpus-level flag encoding:** Flag layers from structurally similar files share distribution properties, making cross-file deduplication potentially significant.

- **Generalised targets:** The current implementation uses bit-clearing ($b^* \in \{0,\ldots,7\}$). The AIM formula supports arbitrary valid targets — mod-N alignment, bitwise masking — that may find additional structure in quantized physical measurement data.

9. Conclusion

We have presented Adaptive Isolating Model (AIM), a parameter-free, lossless, deterministically-terminating algorithm for recursive bit-plane decomposition of arbitrary byte data. The algorithm requires no training data, no shared context, no external dictionary, and no tunable constants. Every decision — which bit to separate, which codec to use for the flag set, what stride to measure, when to halt — is a function of the data and the algorithm.

Version 16 introduces HALT_ANS_STRIDE, a fourth halt condition that intercepts the aligned stream before remap dissolves its inter-symbol correlations. Stride *k* is selected by measuring $H(X_i \mid X_{i-k})$ across eight candidate lags — a domain-free empirical measurement. The optimal cutoff is found by a single bottom-up pass comparing exact sizes. No heuristic, no approximation, no domain knowledge.

The central results are −25.90% versus gzip on a 40.6 MB real-world PCM audio file, and 36.93% of raw size on 527 MB of uncompressed YUV video from a real film. Both verified. Both predicted from first principles. The YUV result closes a 64 MB gap versus the prior best and establishes that HALT_ANS_STRIDE is not a PCM-specific improvement — it is a general mechanism for any data with inter-symbol periodicity at any stride.

The theoretical contribution is unchanged from v14: the termination proof establishes that the symbol space halves at each remap, bounding recursion at eight depths for byte data without any explicit depth limit parameter. HALT_ANS_STRIDE fires at or before the natural terminal and does not affect this proof.

The Huffman analogy, stated precisely: Huffman formalized the optimal parameter-free algorithm for symbol-frequency structure. AIM claims the analogous position for bit-plane and inter-symbol stride structure — a class of structure that is present in all structured byte data, exploited only in domain-specific forms by existing methods, and accessible universally by the sweep-and-clear architecture extended with stride-k conditional entropy measurement. Whether AIM is optimal for that class remains an open question.

That it is universal and parameter-free is established here.

References

Ziv, J. and Lempel, A. (1977). A universal algorithm for sequential data compression. IEEE Transactions on Information Theory, 23(3):337–343.

Ziv, J. and Lempel, A. (1978). Compression of individual sequences via variable-rate coding. IEEE Transactions on Information Theory, 24(5):530–536.

Huffman, D. A. (1952). A method for the construction of minimum-redundancy codes. Proceedings of the IRE, 40(9):1098–1101.

Rissanen, J. and Langdon, G. G. (1979). Arithmetic coding. IBM Journal of Research and Development, 23(2):149–162.

Duda, J. (2009). Asymmetric numeral systems. arXiv:0902.0271.

Deutsch, P. (1996). DEFLATE compressed data format specification version 1.3. RFC 1951, IETF.

Mallat, S. G. (1989). A theory for multiresolution signal decomposition: the wavelet representation. IEEE Transactions on Pattern Analysis and Machine Intelligence, 11(7):674–693.

Taubman, D. S. and Marcellin, M. W. (2002). JPEG 2000: Image Compression Fundamentals, Standards and Practice. Kluwer Academic Publishers.

Kolmogorov, A. N. (1965). Three approaches to the quantitative definition of information. Problems of Information Transmission, 1(1):1–7.

Li, M. and Vitányi, P. (2008). An Introduction to Kolmogorov Complexity and Its Applications. Springer, 3rd edition.

Robinson, T. (1994). SHORTEN: Simple lossless and near-lossless waveform compression. Technical Report CUED/F-INFENG/TR.156, Cambridge University Engineering Department.

Welch, T. A. (1984). A technique for high-performance data compression. Computer, 17(6):8–19.

Appendix A: Availability

Artifact	Description
aim_v15.py	Python reference implementation. All 7 flag formats, HALT_ANS_STRIDE, recursive and CAIM modes, full test suite. No dependencies beyond Python standard library and zlib.
aim_v16.c	C reference implementation. Compiles with: gcc -O3 -o aim aim_v16.c -lz -lm Dependency: zlib only. Zero warnings at -Wall -Wextra.
AIM_Specification_v16.docx	Complete wire format specification including rANS codec, stride selection algorithm, and HALT_ANS_STRIDE wire format. Sufficient to implement a correct decoder in any language without reference to source files.

Both implementations are wire-compatible. A file encoded by the Python implementation decodes correctly with the C implementation and vice versa, verified across all test cases with SHA-256 integrity checking.

Appendix B: Flag Format Detail

Format	ID	Wire structure	Applicability
GAP	0	first_pos (varbyte) + successive_gaps (varbyte each)	Sparse: density < ~5%
BITSET	1	dense bit vector, MSB first, length = ceil(n/8) bytes	General; always applicable
ELIAS-FANO	2	EF encoding of sorted integer sequence	Monotone with clustered gaps
RLE	3	Run-length encoding on bitset representation	Alternating runs
HUFFMAN	4	lengths[256] + n_bits (4B BE) + packed bits MSB-first	Moderate density with byte repetition
LZ77	5	orig_n (4B) + n_tokens (4B) + flag-grouped token stream	Positional repetition in flag stream
LZ77+HUFF	6	LZ77 tokens Huffman-coded (= DEFLATE / gzip)	Large flag sets with mixed structure